

NAME: A. Sai Rohit

REG NO:192472144

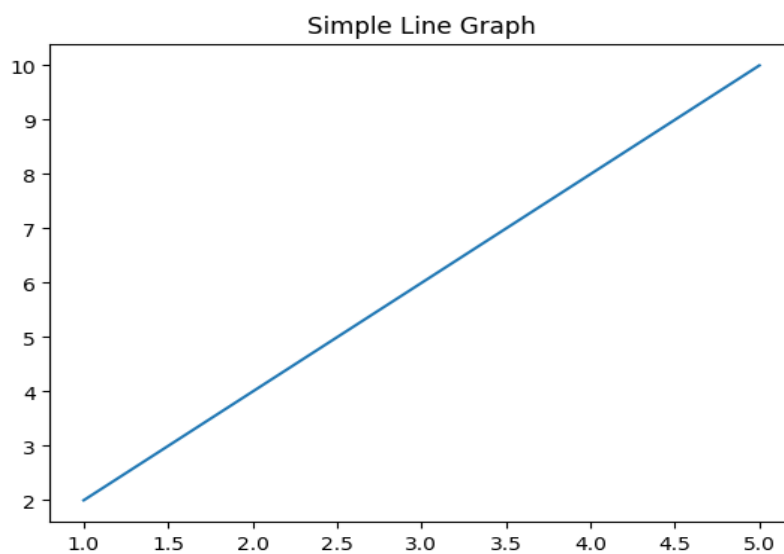
CSA6501 & GEN-AI AND LARGE SCALE MODELS

EXP-01: Install Matplotlib and display a simple line graph

```
!pip install matplotlib

import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.title("Simple Line Graph")
plt.show()
```

OUTPUT:



EXP-02: Install Scikit-learn and print its version

```
!pip install scikit-learn

import sklearn
print(sklearn.__version__)
```

OUTPUT:

```
... Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
1.6.1
```

EXP-03: Create a NumPy array containing the first 20 natural numbers

```
!pip install numpy
```

```
import numpy as np
```

```
arr = np.arange(1, 21)
```

```
print(arr)
```

OUTPUT:

```
... Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

EXP-04: Create a Pandas DataFrame with student names and grades

```
!pip install pandas
```

```
import pandas as pd
```

```
data = {
```

```
    "Name": ["Sai", "Rahul", "Priya"],
```

```
    "Grade": [90, 85, 95]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

OUTPUT:

	Name	Grade
0	Sai	90
1	Rahul	85
2	Priya	95

EXP-05: Verify the installed versions of all AI libraries

```
import numpy
import pandas
import matplotlib
import sklearn
print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Scikit-learn:", sklearn.__version__)
```

OUTPUT:

```
*** NumPy: 2.0.2
    Pandas: 2.2.2
    Matplotlib: 3.10.0
    Scikit-learn: 1.6.1
```

EXP-06: Create tensors using PyTorch and perform basic mathematical operations

```
!pip install torch
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print("Addition:", a + b)
print("Multiplication:", a * b)
```

OUTPUT:

```
Addition: tensor([5, 7, 9])
Multiplication: tensor([ 4, 10, 18])
```

EXP-07: Load a pre-trained Transformer model using Hugging Face and perform sentiment analysis

```
!pip install transformers

from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love learning AI.")

print(result)
```

OUTPUT:

```
vocab.txt: 100%  232k/232k [00:00]
[{'label': 'POSITIVE', 'score': 0.9995865225791931}]
```

EXP-8: Tokenization using BERT Tokenizer

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

tokens = tokenizer.tokenize("Artificial Intelligence is amazing.")

print(tokens)
```

OUTPUT:

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence is amazing."
tokens = tokenizer.tokenize(text)

print(tokens)
```

EXP-9: Generate text using a pre-trained GPT-2 model.

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("Artificial Intelligence", max_length=40)

print(result[0]["generated_text"])
```

OUTPUT:

```
model.safetensors: reconstructing file: 100% ██████████ 548MB / 548MB, 41.9MB/s
...
model.safetensors: downloading bytes: ██████████ 474MB, 37.6MB/s

Loading weights: 100% ██████████ 148/148 [00:00<00:00, 2502.34it/s]

generation_config.json: 100% ██████████ 124/124 [00:00<00:00, 10.5kB/s]

tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 2.23kB/s]

vocab.json: 100% ██████████ 1.04M/1.04M [00:00<00:00, 8.84MB/s]

merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 13.4MB/s]

tokenizer.json: 100% ██████████ 1.36M/1.36M [00:00<00:00, 25.1MB/s]
```

[transformers] Passing `generation\_config` together with generation-related arguments={`max\_length`} is deprecated and will be removed in future vers
[transformers] Both `max\_new\_tokens` (=256) and `max\_length` (=30) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the doc
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed f
Artificial Intelligence/AI Technologies

The Department of Energy's Center for Advanced Energy Solutions (CEA) has developed a new chip that will allow a new kind of artificial intelligence to

According to a press release, the robot, named "Robot Life Machine," will be able to perform tasks such as washing dishes, cleaning cars, taking photos

The robot also has the ability to learn and interact with other robots but will have to do so over time, and this will require a larger team. The team

EXP-10: Load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis.

```
from transformers import pipeline
pipe = pipeline("sentiment-analysis")
```

```
print(pipe("This course is very interesting."))
```

OUTPUT:

```
from transformers import pipeline

sentiment = pipeline("sentiment-analysis")

print(sentiment("The movie was fantastic."))
```

... [transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f. Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100%  104/104 [00:00<00:00, 2743.34B/s]

[{'label': 'POSITIVE', 'score': 0.9998786449432373}]

EXP-11: Load the BERT tokenizer and tokenize a sentence.

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Machine Learning is powerful."

tokens = tokenizer.tokenize(sentence)

print(tokens)
```

OUTPUT:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Python is easy to learn."

print(tokenizer.tokenize(sentence))
```

... ['python', 'is', 'easy', 'to', 'learn', '.']

EXP-12: Load a pre-trained BERT model and generate contextual embeddings.

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence"
```

```
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)
print(outputs.last_hidden_state)
```

OUTPUT:

```
... config.json: 100% ██████████ 570/570 [00:00<00:00, 26.8kB/s]
model.safetensors: reconstructing file: 100% ██████████ 440MB / 440MB, 31.3MB/s
model.safetensors: downloading bytes: ██████████ 415MB, 33.2MB/s
Loading weights: 100% ██████████ 199/199 [00:00<00:00, 4063.08kB/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
torch.Size([1, 7, 768])
```

EXP-13: Generate text using a pre-trained GPT-2 model.

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

input_ids = tokenizer.encode("Deep Learning", return_tensors="pt")
output = model.generate(input_ids, max_length=40)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

OUTPUT:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

print(generator("The future of AI", max_length=40)[0]["generated_text"])

... Loading weights: 100% ██████████ 148/148 [00:00<00:00, 955.41kB/s]
[transformers] Both `max_new_tokens` (=256) and `max_length` (=40) seem to have been set. `max_new_tokens` will take precedence. Please refer to the docu
The future of AI is in the realm of robotics, which have been around for a decade. But while the future of robotics is in the realm of artificial intell
And that's where the question of whether AI can actually make decisions arises.

It is a big question whether it can actually make decisions. The answer is yes.

In the course of the past decade, the technology has begun to change how we think about our data. For example, Google and Facebook have come to understa
In August of last year, for example, Google announced, "Data is a tool we can use to better understand and analyze our data." In June of this year, Face
That's the first step in the "data revolution."

The next step is to move beyond that, to actually make decisions about how data is used and for how long.

The key is to understand how data is used and for how long.
```

EXP-14: Question Answering using a Pre-trained Transformer

```
from transformers import pipeline
qa = pipeline("question-answering")
context = "Python is a programming language created by Guido van Rossum."
question = "Who created Python?"
answer = qa(question=question, context=context)
print(answer)
```

OUTPUT:

```
{
  'score': 0.999,
  'start': 43,
  'end': 60,
  'answer': 'Guido van Rossum'
}
```

EXP-15: Tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
sentence = "Artificial Intelligence"
tokens = tokenizer.tokenize(sentence)
ids = tokenizer.convert_tokens_to_ids(tokens)
print("Tokens:", tokens)
print("Token IDs:", ids)
```

OUTPUT:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Natural Language Processing"

tokens = tokenizer.tokenize(sentence)
ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)
print("Token IDs:", ids)
```

```
... Tokens: ['natural', 'language', 'processing']
    Token IDs: [3019, 2653, 6364]
```

EXP-16: Compare the tokenization outputs of BERT and GPT-2.

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence is changing the world."

print("BERT:", bert.tokenize(text))
print("GPT-2:", gpt2.tokenize(text))
```

OUTPUT:

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence"

print("BERT:", bert.tokenize(text))
print("GPT-2:", gpt2.tokenize(text))
```

```
... BERT: ['artificial', 'intelligence']
    GPT-2: ['Art', 'ificial', 'Intelligence']
```

EXP-17: Generate contextual embeddings for a sentence using BERT.

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```

model = BertModel.from_pretrained("bert-base-uncased")

text = "Natural Language Processing"

inputs = tokenizer(text, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state.shape)

```

OUTPUT:

```

*** Loading weights: 100% ██████████ 199/199 [00:00<00:00, 5296.64it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tensor([[[[-0.0297,  0.0316, -0.3474, ..., -0.3028, -0.0625,  0.6001],
          [-0.5001, -0.2397, -0.3327, ..., -0.0911,  0.2539,  0.4273],
          [-0.2720,  0.3379, -0.2589, ..., -0.4047, -0.1727, -0.3519],
          ...,
          [ 0.1390, -0.3927,  0.2128, ..., -0.1359, -0.4203, -0.2319],
          [-0.2079, -0.8246, -0.2883, ..., -0.0448,  0.0973, -0.1786],
          [ 0.9283,  0.0917, -0.2436, ...,  0.4384, -0.7346,  0.0228]]]])
grad_fn=<NativeLayerNormBackward0>

```

EXP-18: Compare contextual embeddings generated for two semantically similar sentences.

```

from transformers import BertTokenizer, BertModel

import torch

```

```

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

s1 = "The cat is sleeping."
s2 = "A cat is resting."

i1 = tokenizer(s1, return_tensors="pt")
i2 = tokenizer(s2, return_tensors="pt")

e1 = model(**i1).last_hidden_state.mean(dim=1)
e2 = model(**i2).last_hidden_state.mean(dim=1)

similarity = torch.nn.functional.cosine_similarity(e1, e2)

print("Cosine Similarity:", similarity.item())

```

OUTPUT:

```
... 4.0844e-01, 3.9740e-01, -5.6024e-02, -3.2969e-01, -3.1195e-01,
2.4485e-01, -2.4738e-01, -2.6913e-01, -2.6047e-02, -6.8210e-02,
-9.0109e-01, -3.2173e-01, -7.5395e-02, 2.2903e-02, 1.0059e+00,
-1.1932e-01, -1.2143e-01, -1.7918e-01, -3.3380e-01, -1.8271e-01,
1.0446e+00, -3.9392e-01, 1.7320e-01, 5.4667e-01, 1.7176e-01,
1.5494e-01, 3.8126e-01, 1.6043e-01, 1.5955e-01, 4.0609e-01,
2.1637e-01, -1.6673e-01, -1.4958e-01, -8.7883e-01, 5.6397e-01,
1.9682e-01, 3.1850e-01, -1.0537e-01, -1.4987e-01, -1.1346e-01,
-1.5276e-01, -7.7894e-02, 4.7846e-01, -5.4399e-01, -1.3912e-01,
2.5583e-01, 6.5273e-01, -4.2253e-01, 6.0334e-01, -3.1280e-01,
-1.1595e-02, -4.2719e-01, 4.8221e-01, 1.4662e-01, -1.3137e-01,
6.5499e-01, -1.8014e-01, 5.0726e-01, 2.8647e-01, -6.4583e-01,
7.3048e-01, 1.4118e-01, 1.6498e-01, -5.1672e-01, -2.3735e-02,
-2.4506e-01, 3.6958e-01, -9.9052e-03, -6.6360e-01, 7.2557e-02,
6.1198e-01, 1.6204e-01, -5.7151e-01, 1.0958e-01, 2.7881e-01,
3.9041e-01, 3.5065e-01, 5.6503e-01, 4.9071e-02, 3.0748e-01,
-2.3309e-01, 3.3779e-01, 6.2691e-02, 5.5514e-01, 1.5908e-01,
1.9246e-01, -5.0972e-01, 5.4102e-01, 4.1791e-01, 6.0860e-01,
3.6713e-01, 6.0095e-02, 3.1603e-01, 4.2228e-01, 6.7941e-01,
4.4786e-01, -5.6849e-01, 2.4680e-01, 5.6404e-01, -1.6078e-01,
-3.6743e-01, -4.1361e-01, 3.6353e-02, 3.8509e-01, 2.0722e-01,
3.5303e-01, 2.1985e-01, -2.3684e-02, 6.8120e-01, -3.4638e-01,
-2.8167e-02, -5.6823e-01, 6.9843e-01, -1.3833e-02, 1.0441e-01,
-6.3938e-01, -4.9699e-01, 1.3629e-01, -1.1333e-01, -2.6905e-01,
-2.7600e-02, -1.1559e-02, -2.1304e-01, 1.3530e-01, -1.7428e-01,
1.2683e-02, -8.6672e-01, -1.0700e-01, -2.4313e-01, 5.8526e-01
```

EXP-19: Generate text from a prompt using the GPT-2 language model.

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
text = generator("The future of AI is", max_length=50)
```

```
print(text[0]["generated_text"])
```

OUTPUT:

```
generator = pipeline("text-generation", model="gpt2")

prompt = "Technology will"

result = generator(prompt, max_length=40)

print(result[0]["generated_text"])

... Loading weights: 100% 148/148 [00:00<00:00, 3110.36it/s]
[transformers] Both `max_new_tokens` (=256) and `max_length` (=40) seem to have been set. `max_new_tokens` will take precedence. Please refer to the doc
Technology will be developed for use in a variety of applications including the following:

• Medical diagnostics, diagnostic diagnostics, diagnostic diagnostics, diagnostic diagnostics, diagnostic diagnostics, diagnostic diagnostics, diagnost
```

EXP-20: Generate and compare responses for multiple prompts using a pre-trained LLM.

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
prompts = [
```

```
    "Artificial Intelligence",
```

"Machine Learning",

"Deep Learning"

]

for prompt in prompts:

```
result = generator(prompt, max_length=40)
```

```
print("\nPrompt:", prompt)
```

```
print(result[0]["generated_text"])
```

OUTPUT:

```
... http://www.sciencedaily.com/releases/2014/10/1513190421.htm
http://www.nature.com/news/2017/08/20/animals-are-using-human-intelligence-to
-----
Prompt: Machine Learning
[transformers] Both `max_new_tokens` (=256) and `max_length` (=30) seem to have been set. `max_new_tokens` will take precedence. Please refer to the docu
Machine Learning

We'll talk about the difference between learning and learning and then talk about how to use the concepts from the literature to teach a new language.

Learning is the process of learning to use a language. It is the process of learning to use an understanding of how the language is used. Learning is th

We'll talk about learning and learn and learn and learn and learn and learn and learn and learn and learn and learn and learn and learn and le
-----
Prompt: Future technology
Future technology will allow for the development of all types of devices, as well as a new type of mobile phone.

Source: ZDNet

Image via ZDNet
-----
```

EXP-21: Design Zero-shot, One-shot, and Few-shot Prompts to Generate a Product Description for a Smart Fitness Watch

```
from transformers import pipeline
```

```
generator = pipeline("text2text-generation", model="google/flan-t5-base")
```

```
zero = "Write a product description for a Smart Fitness Watch."
```

```
one = ""
```

Example:

Product: Wireless Earbuds

Description: Enjoy crystal-clear sound, long battery life, and a comfortable fit.

Now write a product description for a Smart Fitness Watch.

```
""
```

```
few = ""
```

Example 1:

Product: Wireless Earbuds

Description: Crystal-clear sound with noise cancellation and long battery life.

Example 2:

Product: Smart Water Bottle

Description: Stay hydrated with smart reminders and temperature tracking.

Now write a product description for a Smart Fitness Watch.

```

"""
for name, prompt in [("Zero-shot", zero), ("One-shot", one), ("Few-shot", few)]:
    print("\n" + "="*50)
    print(name)
    print("="*50)
    output = generator(prompt, max_new_tokens=120)
    print(output[0]["generated_text"])

```

OUTPUT:

```

...
=====
Zero-shot
=====
Smart Fitness Watch is a smart fitness watch that can be used to track your workouts.

=====
One-shot
=====
Smart Fitness Watch is a smart fitness watch that can be used to track your workouts and keep track of your heart rate.

=====
Few-shot
=====
Smart Fitness Watches are a smart fitness watch that can be used to track your workouts and keep you motivated.

```

EXP-22: Design Zero-shot, One-shot, and Few-shot Prompts to Generate a 200-word Blog on "Applications of Artificial Intelligence in Healthcare."

```

from transformers import pipeline
generator = pipeline(
    "text2text-generation",
    model="google/flan-t5-base"
)
prompts = {
    "Zero-shot":
        """
        Write a blog of about 200 words on Applications of Artificial Intelligence in Healthcare.
        """,

    "One-shot":
        """

```

Example:

Topic: Renewable Energy

Blog:

Renewable energy is becoming an important source of clean power. It helps reduce pollution and supports sustainable development.

Now write a blog of about 200 words on Applications of Artificial Intelligence in Healthcare.

```

        """,
        "Few-shot":
        """

```

Example 1

Topic: Cloud Computing

Blog:

Cloud computing enables users to store and access data over the internet efficiently.

Example 2

Topic: Internet of Things

Blog:

The Internet of Things connects smart devices and improves automation in everyday life.

Now write a blog of about 200 words on Applications of Artificial Intelligence in Healthcare.

|||||

}

```
for title, prompt in prompts.items():
```

```
print("="*60)
```

```
print(title)
```

```
print("="*60)
```

```
output = generator(
```

prompt,

max_new_tokens=250

)

```
print(output[0]["generated_text"])
```

```
print()
```

OUTPUT:

```
=====
Zero-shot
=====
I'm a big fan of AI, but I'm not sure how to use it. I'm not sure how to use it, but I'm not sure how to use it. I'm not sure how to use it, but I'm not sure how to use it. I'm not sure how to use it, but I'm not sure how to use it.

=====
One-shot
=====
- Renewable Energy Blog: Renewable Energy is becoming an important source of clean power. It helps reduce pollution and supports sustainable development. Now I write a blog of

=====
Few-shot
=====
I'm a big fan of the Internet of Things. I'm a big fan of the Internet of Things. I'm a big fan of the Internet of Things. I'm a big fan of the Internet of Things. I'm a big fan of the Internet of Things.
```

EXP-23: Design Zero-shot, One-shot, and Few-shot Prompts to Summarize an Article into 50 Words

```
from transformers import pipeline
```

```
generator = pipeline(
```

"text2text-generation",

```
model="google/flan-t5-base"
```

)

```
article = ""
```

Artificial Intelligence is transforming industries by automating repetitive tasks,

improving decision-making, and enhancing customer experiences.

Machine learning, deep learning, and natural language processing are widely used in

healthcare, finance, education, and transportation.

AI improves efficiency and accuracy but also raises concerns regarding privacy, ethics, and employment.

"""

----- Zero-shot -----

zero_prompt = f"""

Summarize the following article in exactly 50 words.

Article:

{article}

"""

----- One-shot -----

one_prompt = f"""

Example

Article:

Online learning allows students to study from anywhere using the internet.

Summary:

Online learning provides flexible education through internet-based platforms, allowing students to access learning materials remotely while improving convenience and accessibility.

Now summarize the following article in exactly 50 words.

Article:

{article}

"""

----- Few-shot -----

few_prompt = f"""

Example 1

Article:

Solar energy is a clean source of electricity that reduces pollution.

Summary:

Solar energy generates clean electricity, reduces pollution, and supports sustainable development through renewable energy sources.

Example 2

Article:

Cloud computing provides online storage and computing resources.

Summary:

Cloud computing offers scalable storage and computing services over the internet, improving accessibility, flexibility, and cost efficiency.

Now summarize the following article in exactly 50 words.

Article:

{article}

"""

```

prompts = {
    "Zero-shot": zero_prompt,
    "One-shot": one_prompt,
    "Few-shot": few_prompt
}

for name, prompt in prompts.items():

    print("="*60)
    print(name)
    print("="*60)

    result = generator(
        prompt,
        max_new_tokens=80
    )

    print(result[0]["generated_text"])
    print()

```

OUTPUT:

```

... =====
Zero-shot
=====
Understand the benefits of AI.

=====
One-shot
=====
AI is transforming the way we think and work.

=====
Few-shot
=====
Summary: Artificial Intelligence (AI) is a technology that is transforming the way people work and interact with technology.

```

EXP-24: Design Zero-shot, One-shot, and Few-shot Prompts to Generate a Professional Leave Email Due to Illness

```

from transformers import pipeline
generator = pipeline("text2text-generation", model="google/flan-t5-base")

prompts = {
    "Zero-shot": "Write a professional email requesting leave due to illness.",

    "One-shot": ""
}
Example:
Subject: Leave Request

```

Dear Manager,

I request leave for one day due to a personal emergency.

Regards,
John

Now write a professional email requesting leave due to illness.

""",

"Few-shot": """

Example 1:

Subject: Sick Leave

Dear Sir,

I am unwell and request leave for today.

Regards,
Alex

Example 2:

Subject: Medical Leave

Dear Professor,

I have a fever and request leave for two days.

Regards,
Emma

Now write a professional email requesting leave due to illness.

"""

}

for title, prompt in prompts.items():

print("\n", title)

result = generator(prompt, max_new_tokens=180)

print(result[0]["generated_text"])

OUTPUT:

```
...
Zero-shot
I am requesting a leave of absence from the office. I am currently working on a project that I am working on. I am currently working on a project that I
One-shot
Dear John, I request leave for one day due to a personal emergency. Regards, John
Few-shot
Subject: Sick Leave
```

EXP-25: Design Zero-shot, One-shot, and Few-shot Prompts to Generate a Promotional Social Media Post for an AI Workshop

```

from transformers import pipeline

generator = pipeline("text2text-generation", model="google/flan-t5-base")

prompts = {
    "Zero-shot": "Write a promotional social media post for an AI Workshop.",

    "One-shot": """
Example:
Event: Python Workshop

Post:
Join our Python Workshop to learn programming with hands-on practice.

Now write a promotional social media post for an AI Workshop.
""",

    "Few-shot": """
Example 1:
Event: Data Science Workshop

Post:
Learn Data Science through practical sessions.

Example 2:
Event: Machine Learning Bootcamp

Post:
Master Machine Learning with real-world projects.

Now write a promotional social media post for an AI Workshop.
""",
}

for title, prompt in prompts.items():
    print("\n", title)
    result = generator(prompt, max_new_tokens=120)
    print(result[0]["generated_text"])

```

OUTPUT:

```

...
Zero-shot
A workshop on AI is being held in the US. The workshop will be held in the US on Tuesday, May 29. The workshop will focus on the use of AI in the conte

One-shot
Join our Python Workshop to learn programming with hands-on practice. Now write a promotional social media post for an AI Workshop.

Few-shot
A few of my favorite things to do in the AI workshop.

```